Bases de données avancées

GLO-4035

Automne 2024

Réalisé par

Zack Dufresne (536 899 030)

Charles-Étienne Dumont (536 960 589)

Charles Brassard (536 783 602)

Christian Fosso (536 904 549)

Étude de faisabilité

Travail présenté à

Eloïse Prévôt

1er octobre 2024

Stratégie d’acquisition des données

Présentation de la ville

La ville que nous avons choisie pour réaliser ce travail est Rimouski. Considérée comme l’une des plus grandes villes de la région du Bas-Saint-Laurent, elle se distingue par sa proximité avec le fleuve Saint-Laurent. On y trouve ainsi des pistes cyclables, notamment la célèbre promenade de la mer, qui longe le fleuve et le centre-ville. La ville abrite de nombreux restaurants de tous types, dont plusieurs spécialisés dans les fruits de mer en raison de sa proximité avec le fleuve Saint-Laurent et la Gaspésie.

Source de données

Les données géospatiales utilisées pour ce projet seront récupérées de OpenStreetMap (OSM). C’est une base de données open source qui fournit beaucoup d’informations détaillées sur les routes et points d’intérêts à partir d’une localisation. Grâce au langage de programmation utilisé qui sera présenté plus bas, nous pouvons utiliser la bibliothèque “osmnx” pour récupérer les coordonnées des restaurants de la ville.Nous allons par la suite récupérer à partir de OpenRouteService les itinéraires optimales à vélo entre deux coordonnées géospatiales, qui combiné avec OSM nous permettra d’obtenir toutes les données nécessaires.

Méthode d’obtention des données

Pour obtenir les données sur les restaurants, nous utiliserons du code similaire à celui présenté à l’Annexe 1, en appliquant bien sûr un champ “amenity” à restaurant et en ajustant la ville à Rimouski. Pour obtenir les données des itinéraires de pistes cyclables de la ville, nous utiliserons une requête vers l’API de OpenRouteService similaire à celle présentée en Annexe 2.

Analyse des données

Les données sur les restaurants récupérées seront sous la forme d’un GeoDataFrame à partir duquel il est très simple d’extraire les données géospatiales requise pour la suite ou même d’effectuer des opérations de calculs sur ces données. Pour les itinéraires à vélo, la requête présentée en Annexe 2 (où la clé API est omise intentionnellement) retourne un objet JSON qui contient toutes les informations d’itinéraires suffisantes.

Exemples de données

Des exemples de données sont présentés dans les Annexes 3, 4, 5, 6, 7, 8, 9, 10, 11

Technologies utilisées

Langage de programmation

Pour notre travail longitudinal nous avons décidé d'utiliser Python comme langage de programmation. Ce choix a été adopté notamment pour la simplicité et l'accessibilité de ce dernier qui contribueront à la maintenabilité du système. En ce sens, nous considérons que Python est un outil populaire qui possède une vaste communauté et une documentation abondante ce qui permettra à tous une meilleure autonomie. Par ailleurs, celui-ci permet d'obtenir du code lisible, clair et intuitif ce qui facilitera la compréhension et la communication au sein de l'équipe. De plus, Python peut aussi contribuer à la fiabilité du système. D'une part, il offre un moyen robuste de gérer les exceptions grâce à des blocs try-except ce qui permettra un meilleur contrôle des erreurs. D'une autre part, il permet de détecter les problèmes rapidement et de les corriger au fur et à mesure en effectuant des tests automatisés grâce au framework pytest.

À noter que le langage JavaScript sera utilisé pour implémenter une partie de notre API. Celui-ci permet notamment une bonne réutilisation du code ce qui contribuera à la maintenabilité du système. Pour en ajouter, il permet une bonne fiabilité pour les mêmes raisons que Python et par sa gestion des erreurs avec "promises".

Base de données

Les types de bases de données que nous avons choisis pour effectuer notre travail longitudinal sont les bases de données orientées document et graphe. Tout d'abord, nous avons choisi le type document car il permet efficacement de stocker les données dans un format qui peut facilement être modifié et qui n'a pas à être constant si les données ne le sont pas elles-mêmes. Ce type est donc bien extensible, ce qui sera un avantage si on doit ajouter de nouvelles données, dont certaines ne seront peut-être pas dans le même format que celles présentes dans la base de données. Dans notre cas, ce type sera utilisé pour stocker les informations sur les restaurants. Nous avons également utilisé le type graphe, car celui-ci permet efficacement de représenter les relations *many-to-many* comme dans notre cas, les réseaux de pistes cyclables. Similairement aux bases de données de types document, celles de types graphes sont extensibles, ce qui sera utile si on doit ajouter de nouveaux types de relations ou étendre les réseaux de pistes cyclables.

Annexe 1

Exemple de code Python pour récupérer des “amenity” restaurant de la ville Rimouski

```python
import osmnx as ox

tags = { "amenity": "restaurant", "highway": "cycleway" }

gdf = ox.features_from_place("Rimouski, Quebec, Canada", tags)
```

Annexe 2

Exemple de requête envoyé vers l’API de OpenRouteService pour l’itinéraire en vélo entre deux points (dans notre cas, les points seront des coordonnées de restaurants, la clé est omise par standard de confidentialité)

```
GET https://api.openrouteservice.org/v2/directions/cycling-regular?api_key=api-key&start=8.681495,49.41461&end=8.687872,49.420318
```

Annexe 3

Exemple de structure d’un GeoDataFrame créé par le code de l’Annexe 1 qui contiendra les restaurants de la ville (généré par intelligence artificielle ChatGPT 4o)

	osmid	name	geometry	amenity	addr:street
0	123456789	Chez Gourmet	POINT (-68.52 48.44)	restaurant	Rue Principale
1	987654321	Bistro du Port	POINT (-68.53 48.45)	restaurant	Avenue de la Mer

Annexe 4

Exemple d'objet JSON retourné par une requête de format similaire à celle présenté à l'Annexe 2 (les champs distance en mètres et duration en secondes correspondent aux détails du trajet global, le champ concaténé "steps" énumère les différentes étapes du trajet avec instruction de déplacement progressive)


```
{
    "type": "FeatureCollection",
    "bbox": [8.681423, 49.414599, 8.690123, 49.420514],
    "features": [
      {
        "bbox": [8.681423, 49.414599, 8.690123, 49.420514],
        "type": "Feature",
        "properties": {
          "segments": [
            {
              "distance": 1408.8,
              "duration": 281.9,
              "steps": [ ···
              ]
            }
          ],
          "way_points": [0, 23],
          "summary": {
            "distance": 1408.8,
            "duration": 281.9
          }
        },
```


Annexe 5

Exemple d'erreur de collecte de données, la ville de Los Angeles n'étant pas au Québec

```
import osmnx as ox

tags = { "amenity": "restaurant", "highway": "cycleway" }

gdf = ox.features_from_place("Los Angeles, Quebec, Canada", tags)
```

Annexe 6

Exemple d'erreur de collecte de données en omettant l'emplacement désiré

```
import osmnx as ox

tags = { "amenity": "restaurant", "highway": "cycleway" }

gdf = ox.features_from_place(tags)
```

Annexe 7

Exemple d'erreur de collecte de données, récupérant les banques au lieu des restaurants, en plus d'une faute dans le nom de la ville

```
import osmnx as ox

tags = { "amenity": "bank", "highway": "cycleway" }

gdf = ox.features_from_place("Rimousky, Quebec, Canada", tags)
```

Annexe 8

Exemple d'erreur d'extraction de données, le mode de transport est la voiture, ce qui ne correspond pas à ce que l'on souhaite

GET https://api.openrouteservice.org/v2/directions/driving-car?api_key=your-api-key&start=8.681495,49.41461&end=8.687872,49.420318

Annexe 9

Exemple d'erreur d'extraction de données, la clé API est omise et laissé à nulle, ne permettant pas la récupération des données

GET https://api.openrouteservice.org/v2/directions/cycling-regular?api_key=null&start=8.681495,49.41461&end=8.687872,49.420318

Annexe 10

Exemple d'erreur de données, utilisant des coordonnées géospatiales invalide en dehors des limites conventionnellement établies

GET https://api.openrouteservice.org/v2/directions/cycling-regular?api_key=api-key&start=95.000000,-195.000000&end=8.687872,49.420318

Annexe 11

Exemple d'erreur de formatage de données, utilisant de virgules "," pour les coordonnées de départ à la place de points "."

GET https://api.openrouteservice.org/v2/directions/cycling-regular?api_key=api-key&start=8,681495,49,41461&end=8.687872,49.420318

Bibliographie

[1] OpenRouteService. (n.d.). *API Documentation*. https://openrouteservice.org/dev/#/api-docs

[2] Babaskïn, E. (n.d.). *Data Quality: Les types d'erreurs*. LinkedIn. https://www.linkedin.com/pulse/data-quality-les-types-derreurs-eugene-babask%C3%AFn/

[3] Boeing, G. (n.d.). *OSMnx Examples: Download OSM Geospatial Features*. GitHub. https://github.com/gboeing/osmnx-examples/blob/254b64e0e17f99a16e273b26d782f7a405888731/notebooks/16-download-osm-geospatial-features.ipynb

[4] Boeing, G. (n.d.). *OSMnx User Reference: Features Module*. OSMnx Documentation. https://osmnx.readthedocs.io/en/stable/user-reference.html#module-osmnx.features